

SEAM DDELAY

Giuseppe Silvi

May 6, 2026

author	Giuseppe Silvi
compile`options	-lang cpp -fpga-mem-th 4 -ct 1 -es 1 -mcd 16 -mdd 1024 -mdy 33 -single -ftz 0
delays.lib/name	Faust Delay Library
delays.lib/version	1.2.0
description	Quad speaker-alignment delay (interior c, nextprime quantized)
filename	ddelay.dsp
license	GPL-3.0
maths.lib/author	GRAME
maths.lib/copyright	GRAME
maths.lib/license	LGPL with exception
maths.lib/name	Faust Math Library
maths.lib/version	2.9.0
name	SEAM DDELAY
platform.lib/name	Generic Platform Library
platform.lib/version	1.3.0
vendor	SEAM
version	0.1.0

This document provides a mathematical description of the Faust program text stored in the `ddelay.dsp` file. See the notice in Section ?? (page ??) for details.

1 Mathematical definition of process

The *ddelay* program evaluates the signal transformer denoted by **process**, which is mathematically defined as follows:

1. Output signals y_i for $i \in [1, 4]$ such that

$$y_1(t) = x_1(t - p_1(t))$$

$$y_2(t) = x_2(t - p_1(t))$$

$$y_3(t) = x_3(t - p_1(t))$$

$$y_4(t) = x_4(t - p_1(t))$$

2. Input signals x_i for $i \in [1, 4]$
3. User-interface input signal u_{s1} such that

$$\text{"Distance" (m)} \quad u_{s1}(t) \in [0, 30] \quad (\text{default value} = 0)$$

4. Intermediate signal p_1 such that

$$p_1(t) = \min(32768, \max(0, \text{int}(k_1 \cdot u_{s1}(t))))$$

5. Constant k_1 such that

$$k_1 = 0.00301750150875075 \cdot \min(192000, \max(1, f_s))$$

2 Block diagram of process

The block diagram of `process` is shown on Figure ?? (page ??).

Figure 1: Block diagram of `process`

3 Notice

- This document was generated using Faust version 2.85.5 on May 06, 2026.
- The value of a Faust program is the result of applying the signal transformer denoted by the expression to which the `process` identifier is bound to input signals, running at the f_s sampling frequency.
- Faust (*Functional Audio Stream*) is a functional programming language designed for synchronous real-time signal processing and synthesis applications. A Faust program is a set of bindings of identifiers to expressions that denote signal transformers. A signal s in S is a function mapping¹ times $t \in \mathbb{Z}$ to values $s(t) \in \mathbb{R}$, while a signal transformer is a function from S^n to S^m , where $n, m \in \mathbb{N}$. See the Faust manual for additional information (<http://faust.grame.fr>).
- Every mathematical formula derived from a Faust expression is assumed, in this document, to having been normalized (in an implementation-dependent manner) by the Faust compiler.
- A block diagram is a graphical representation of the Faust binding of an identifier I to an expression E ; each graph is put in a box labeled by I . Subexpressions of E are recursively displayed as long as the whole picture fits in one page.

¹Faust assumes that $\forall s \in S, \forall t \in \mathbb{Z}, s(t) = 0$ when $t < 0$.

- $\forall x \in \mathbb{R}$,

$$\text{int}(x) = \begin{cases} \lfloor x \rfloor & \text{if } x > 0 \\ \lceil x \rceil & \text{if } x < 0 \\ 0 & \text{if } x = 0 \end{cases}.$$

- The `ddelay-mdoc/` directory may also include the following subdirectories:
 - `cpp/` for Faust compiled code;
 - `pdf/` which contains this document;
 - `src/` for all Faust sources used (even libraries);
 - `svg/` for block diagrams, encoded using the Scalable Vector Graphics format (<http://www.w3.org/Graphics/SVG/>);
 - `tex/` for the L^AT_EX source of this document.

4 Faust code listings

This section provides the listings of the Faust code used to generate this document, including dependencies.

Listing 1: `ddelay.dsp`

```

1 declare name      "SEAM DDELAY";
2 declare vendor    "SEAM";
3 declare version   "0.1.0";
4 declare author    "Giuseppe Silvi";
5 declare license   "GPL-3.0";
6 declare description "Quad speaker-alignment delay (interior c, nextprime quantized)";
7 //
8 // DDELAY      Quad Speaker-Alignment Delay
9 //
10 // Pure integer-sample delay for time-aligning loudspeakers in a
11 // sound-reinforcement chain. The user enters the speaker distance
12 // in metres; the plugin converts to samples using the interior
13 // speed of sound (c = 331.4 m/s) and quantizes to the next prime.
14 //
15 // Across multiple plugin instances driving different speakers, the
16 // prime quantization makes the delay lengths mutually incommensurate
17 // the same principle that staggers Schroeder allpass lengths in
18 // reverbs, here applied to acoustic loudspeaker pathing.
19 //
20 // Within one instance the four channels share the same delay
21 // (synchronous). Per-speaker incommensurability comes from running
22 // independent instances on different bus paths.
23 //
24 // FAUST PROTOTYPE (this file, illustrative the C++ plugin
25 // performs the same computation plus the prime quantization):
26 //
27 import("stdfaust.lib");
28 //
29 // Interior speed of sound (matches seam.math.lib::isos)
30 isos = 331.4;
31 //
32 // Distance      samples (rounded down)
33 imt2samp(mt) = int(mt * ma.SR / isos);
34 //

```

```

35 // User control: distance in metres
36 distance = hslider("Distance [unit:m]", 0, 0, 30, 0.01);
37 //
38 // Quad delay, all channels synchronous, no interpolation
39 process = par(i, 4, de.delay (1 << 15, imt2samp(distance)));

```

Listing 2: stdfaust.lib

```

1 //##### stdfaust.lib #####
2 // The purpose of this library is to give access to all the Faust standard libraries
3 // through a series of environments.
4 //#####
5
6 aa = library("aanl.lib");
7 sf = library("all.lib");
8 an = library("analyzers.lib");
9 ba = library("basics.lib");
10 co = library("compressors.lib");
11 db = library("debug.lib");
12 de = library("delays.lib");
13 dm = library("demos.lib");
14 dx = library("dx7.lib");
15 en = library("envelopes.lib");
16 fd = library("fds.lib");
17 fi = library("filters.lib");
18 ho = library("hoa.lib");
19 hy = library("hysteresis.lib");
20 it = library("interpolators.lib");
21 la = library("linearalgebra.lib");
22 ma = library("maths.lib");
23 mi = library("mi.lib");
24 mo = library("motion.lib");
25 ef = library("misceffects.lib");
26 no = library("noises.lib");
27 os = library("oscillators.lib");
28 pf = library("phaflangers.lib");
29 pl = library("platform.lib");
30 pm = library("physmodels.lib");
31 qu = library("quantizers.lib");
32 rm = library("reducemaps.lib");
33 re = library("reverbs.lib");
34 ro = library("routes.lib");
35 sp = library("spats.lib");
36 si = library("signals.lib");
37 so = library("soundfiles.lib");
38 sy = library("synths.lib");
39 ve = library("vaeffects.lib");
40 vl = library("version.lib");
41 wa = library("webaudio.lib");
42 wd = library("wdmodels.lib");

```

Listing 3: delays.lib

```

1 //##### delays.lib #####
2 // Delays library. Its official prefix is 'de'.
3 //
4 // This library provides reusable building blocks for delay-based processing:
5 // single and multi-tap delays, fractional delays and utilities for echo and spatial effects.
6 //
7 // The Delays library is organized into 4 sections:

```

```

8 //
9 // * [Basic Delay Functions] (#basic-delay-functions)
10 // * [Lagrange Interpolation] (#lagrange-interpolation)
11 // * [Thiran Allpass Interpolation] (#thiran-allpass-interpolation)
12 // * [Others] (#others)
13 //
14 // #### References
15 //
16 // * <https://github.com/grame-cncm/faustlibraries/blob/master/delays.lib>
17 //#####
18
19 /*****
20 *****/
21 FAUST library file
22 Copyright (C) 2003-2016 GRAME, Centre National de Creation Musicale
23 -----
24 This program is free software; you can redistribute it and/or modify
25 it under the terms of the GNU Lesser General Public License as
26 published by the Free Software Foundation; either version 2.1 of the
27 License, or (at your option) any later version.
28
29 This program is distributed in the hope that it will be useful,
30 but WITHOUT ANY WARRANTY; without even the implied warranty of
31 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
32 GNU Lesser General Public License for more details.
33
34 You should have received a copy of the GNU Lesser General Public
35 License along with the GNU C Library; if not, write to the Free
36 Software Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA
37 02111-1307 USA.
38
39 EXCEPTION TO THE LGPL LICENSE : As a special exception, you may create a
40 larger FAUST program which directly or indirectly imports this library
41 file and still distribute the compiled code generated by the FAUST
42 compiler, or a modified version of this compiled code, under your own
43 copyright and license. This EXCEPTION TO THE LGPL LICENSE explicitly
44 grants you the right to freely choose the license for the resulting
45 compiled code. In particular the resulting compiled code has no obligation
46 to be LGPL or GPL. For example you are free to choose a commercial or
47 closed source license or any other license if you decide so.
48 *****/
49 *****/
50
51 ma = library("maths.lib");
52 ba = library("basics.lib");
53 si = library("signals.lib");
54 fi = library("filters.lib");
55
56 declare name "Faust Delay Library";
57 declare version "1.2.0";
58
59 //=====Basic Delay Functions=====
60 //=====
61
62 //-----'(de.)delay'-----
63 // Simple 'd' samples delay where 'n' is the maximum delay length as a number of
64 // samples. Unlike the '@' delay operator, here the delay signal 'd' is explicitly
65 // bounded to the interval [0..n]. The consequence is that delay will compile even
66 // if the interval of d can't be computed by the compiler.
67 // 'delay' is a standard Faust function.
68 //
69 // #### Usage
70 //
71 // ''
72 // _ : delay(n,d) : _
73 // ''
74 //
75 // Where:

```

```

76 //
77 // * 'n': the max delay length in samples
78 // * 'd': the delay length in samples (integer)
79
80 // #### Test
81 // '''
82 // de = library("delays.lib");
83 // os = library("oscillators.lib");
84 // delay_test = os.osc(440) : de.delay(44100, 22050);
85 // '''
86 //
87 // TODO: add MBH np2
88 //-----
89 delay(n,d,x) = x @ min(n, max(0,d));
90
91
92 //-----'(de.)fdelay'-----
93 // Simple 'd' samples fractional delay based on 2 interpolated delay lines where 'n' is
94 // the maximum delay length as a number of samples.
95 //
96 // 'fdelay' is a standard Faust function.
97 //
98 // #### Usage
99 //
100 // '''
101 // _ : fdelay(n,d) : _
102 // '''
103 //
104 // Where:
105 //
106 // * 'n': the max delay length in samples
107 // * 'd': the delay length in samples (float)
108 //
109 // #### Test
110 // '''
111 // de = library("delays.lib");
112 // os = library("oscillators.lib");
113 // fdelay_test = os.osc(440) : de.fdelay(44100, 22050.5);
114 // '''
115 //-----
116 fdelay(n,d,x) = delay(n+1,int(d),x)*(1 - ma.frac(d)) + delay(n+1,int(d)+1,x)*ma.frac(d);
117
118
119 //-----'(de.)sdelay'-----
120 // s(mooth)delay: a mono delay that doesn't click and doesn't
121 // transpose when the delay time is changed.
122 //
123 // #### Usage
124 //
125 // '''
126 // _ : sdelay(n,it,d) : _
127 // '''
128 //
129 // Where:
130 //
131 // * 'n': the max delay length in samples
132 // * 'it': interpolation time (in samples), for example 1024
133 // * 'd': the delay length in samples (float)
134 //
135 // #### Test
136 // '''
137 // de = library("delays.lib");
138 // os = library("oscillators.lib");
139 // sdelay_test = os.osc(440) : de.sdelay(44100, 1024, 22050.5);
140 // '''
141 //-----
142 sdelay(n, it, d) = ctrl(it,d),_ : ddi(n)
143 with {

```

```

144 // ddi(n,i,d0,d1)
145 // DDI (Double Delay with Interpolation) : the input signal is sent to two
146 // delay lines. The outputs of these delay lines are crossfaded with
147 // an interpolation stage. By acting on this interpolation value one
148 // can move smoothly from one delay to another. When <i> is 0 we can
149 // freely change the delay time <d1> of line 1, when it is 1 we can freely change
150 // the delay time <d0> of line 0.
151 //
152 // <n> = maximal delay in samples
153 // <i> = interpolation value between 0 and 1 used to crossfade the outputs of the
154 // two delay lines (0.0: first delay line, 1.0: second delay line)
155 // <d0> = delay time of delay line 0 in samples between 0 and <N>-1
156 // <d1> = delay time of delay line 1 in samples between 0 and <N>-1
157 // < > = the input signal we want to delay
158 ddi(n, i, d0, d1) = _<: delay(n,d0), delay(n,d1) : si.interpolate(i);
159
160 // ctrl(it,d)
161 // Control logic for a Double Delay with Interpolation according to two
162 //
163 // USAGE : ctrl(it,d)
164 // where :
165 // <it> an interpolation time (in samples, for example 256)
166 // <d> a delay time (in samples)
167 //
168 // ctrl produces 3 outputs : an interpolation value <i> and two delay
169 // times <d0> and <d1>. These signals are used to control a ddi (Double Delay with
170 // Interpolation).
171 // The principle is to detect changes in the input delay time d, then to
172 // change the delay time of the delay line currently unused and then by a
173 // smooth crossfade to remove the first delay line and activate the second one.
174 //
175 // The control logic has an internal state controlled by 4 elements
176 // <v> : the interpolation variation (0, 1/it, -1/it)
177 // <i> : the interpolation value (between 0 and 1)
178 // <d0> : the delay time of line 0
179 // <d1> : the delay time of line 1
180 //
181 // Please note that the last stage (!,_,_,_) cut <v> because it is only
182 // used internally.
183 ctrl(it, d) = \<(v,ip,d0,d1).(nv, nip, nd0, nd1)
184 with {
185 // interpolation variation
186 nv = ba.if (v!=0.0, // if variation we are interpolating
187 ba.if ((ip>0.0) & (ip<1.0), v, 0), // should we continue or not ?
188 ba.if ((ip==0.0) & (d!=d0), 1.0/it, // if true xfade from d10 to d11
189 ba.if ((ip==1.0) & (d!=d1), -1.0/it, // if true xfade from d11 to d10
190 0)); // nothing to change
191 // interpolation value
192 nip = ip+nv : min(1.0) : max(0.0);
193
194 // update delay time of line 0 if needed
195 nd0 = ba.if ((ip >= 1.0) & (d1!=d), d, d0);
196
197 // update delay time of line 1 if needed
198 nd1 = ba.if ((ip <= 0.0) & (d0!=d), d, d1);
199 } ~ (<_,_,_,_) : (!,_,_,_);
200 };
201
202 //-----'(<de.>prime_power_delays'-----
203 // Prime Power Delay Line Lengths.
204 //
205 // #### Usage
206 //
207 // '
208 // si.bus(N) : prime_power_delays(N,pathmin,pathmax) : si.bus(N);
209 // '
210 //

```

```

211 // Where:
212 //
213 // * 'N': positive integer up to 16 (for higher powers of 2, extend 'primes' array below)
214 // * 'pathmin': minimum acoustic ray length in the reverberator (in meters)
215 // * 'pathmax': maximum acoustic ray length (meters) - think "room size"
216 //
217 // #### Test
218 // '''
219 // de = library("delays.lib");
220 // prime_power_delays_test = de.prime_power_delays(4, 1, 10);
221 // '''
222 //
223 // #### References
224 //
225 // * <https://ccrma.stanford.edu/~jos/pasp/Prime\_Power\_Delay\_Line.html>
226 //-----
227 declare prime_power_delays author "Julius O. Smith III";
228
229 prime_power_delays(N,pathmin,pathmax) = par(i,N,delayvals(i)) with {
230   Np = 16;
231   primes = 2,3,5,7,11,13,17,19,23,29,31,37,41,43,47,53;
232   prime(n) = primes : ba.selector(n,Np); // math.lib
233
234   // Prime Power Bounds [matlab: floor(log(maxdel)./log(primes(53)))]
235   maxdel = 8192; // more than 63 meters at 44100 samples/sec & 343 m/s
236   ppbs = 13,8,5,4, 3,3,3,3, 2,2,2,2, 2,2,2,2; // 8192 is enough for all
237   ppb(i) = ba.take(i+1,ppbs);
238
239   // Approximate desired delay-line lengths using powers of distinct primes:
240   c = 343; // soundspeed in m/s at 20 degrees C for dry air
241   dmin = ma.SR*pathmin/c;
242   dmax = ma.SR*pathmax/c;
243   dl(i) = dmin * (dmax/dmin)^(i/float(N-1)); // desired delay in samples
244   ppwr(i) = floor(0.5+log(dl(i))/log(prime(i))); // best prime power
245   delayvals(i) = prime(i)^ppwr(i); // each delay a power of a distinct prime
246 };
247
248
249 //=====Lagrange Interpolation=====
250 //=====
251
252 //-----'fdelaylti' and '(de.)fdelayltv'-----
253 // Fractional delay line using Lagrange interpolation.
254 //
255 // #### Usage
256 //
257 // '''
258 // _ : fdelaylt[i/v](N, n, d) : _
259 // '''
260 //
261 // Where:
262 //
263 // * 'N=1,2,3,...' is the order of the Lagrange interpolation polynomial (constant numerical
264 //   expression)
265 // * 'n': the max delay length in samples
266 // * 'd': the delay length in samples
267 //
268 // 'fdelaylti' is most efficient, but designed for constant/slowly-varying delay.
269 // 'fdelayltv' is more expensive and more robust when the delay varies rapidly.
270 //
271 // Note: the requested delay should not be less than '(N-1)/2'.
272 //
273 // #### Test
274 // '''
275 // de = library("delays.lib");
276 // os = library("oscillators.lib");
277 // fdelaylti_test = os.osc(440) : de.fdelaylti(3, 44100, 22050.5);
278 // '''

```

```

278 //
279 // #### References
280 //
281 // * <https://ccrma.stanford.edu/~jos/pasp/Lagrange\_Interpolation.html>
282 // * [Fixed-delay case](<https://ccrma.stanford.edu/~jos/Interpolation/
Efficient_Time_Invariant_Lagrange_Interpolation.html)
283 // * [Variable-delay case](<https://ccrma.stanford.edu/~jos/Interpolation/
Time_Varying_Lagrange_Interpolation.html)
284 // * Timo I. Laakso et al., "Splitting the Unit Delay - Tools for Fractional Delay Filter
Design", IEEE Signal Processing Magazine, vol. 13, no. 1, pp. 30-60, Jan 1996.
285 // * Philippe Depalle and Stephan Tassart, "Fractional Delay Lines using Lagrange
Interpolators", ICMC Proceedings, pp. 341-343, 1996.
286 //-----
287 declare fdelaylti author "Julius O. Smith III";
288
289 fdelaylti(N,n,d,x) = delay(n,id,x) <: seq(i,N,section(i)) : !, _
290 with {
291   o = (N-1.00001)/2; // offset to ~center FIR interpolator
292   dmo = d - o; // assumed nonnegative [d > (N-1)/2]
293   id = int(dmo);
294   fd = o + ma.frac(dmo);
295   section(i,x,y) = (x-x') * c(i) <: _,+(y);
296   c(i) = (i - fd)/(i+1);
297 };
298
299 declare fdelayltv author "Julius O. Smith III";
300
301 fdelayltv(N,n,d,x) = sum(i, N+1, delay(n,id+i,x) * h(N,fd,i))
302 with {
303   o = (N-1.00001)/2; // ~center FIR interpolator
304   dmo = d - o; // assumed nonnegative [d > (N-1)/2]
305   id = int(dmo);
306   fd = o + ma.frac(dmo);
307   h(N,d,n) = facs1(N,d,n) * facs2(N,d,n);
308   facs1(N,d,n) = select2(n,1,prod(k,max(1,n),select2(k<n,1,fac(d,n,k))));
309   facs2(N,d,n) = select2(n<N,1,prod(l,max(1,N-n),fac(d,n,l+n+1)));
310   fac(d,n,k) = (d-k)/((n-k)+(n==k));
311   // Explicit formula for Lagrange interpolation coefficients:
312   // h_d(n) = \prod_{\stackrel{k=0}{k \neq n}}^N \frac{d-k}{n-k}, n=0:N
313 };
314
315 //-----'(de.)fdelay[N]'-----
316 // For convenience, 'fdelay1', 'fdelay2', 'fdelay3', 'fdelay4', 'fdelay5'
317 // are also available where 'N' is the order of the interpolation, built using 'fdelayltv'.
318 //-----
319 declare fdelay1 author "Julius O. Smith III";
320 declare fdelay2 author "Julius O. Smith III";
321 declare fdelay3 author "Julius O. Smith III";
322 declare fdelay4 author "Julius O. Smith III";
323 declare fdelay5 author "Julius O. Smith III";
324
325 fdelay1 = fdelayltv(1);
326 fdelay2 = fdelayltv(2);
327 fdelay3 = fdelayltv(3);
328 fdelay4 = fdelayltv(4);
329 fdelay5 = fdelayltv(5);
330
331 //=====Thiran Allpass Interpolation=====
332 // Thiran Allpass Interpolation.
333 //
334 // #### References
335 //
336 // * <https://ccrma.stanford.edu/~jos/pasp/Thiran\_Allpass\_Interpolators.html>
337 //=====
338
339 //-----'(de.)fdelay[N]a'-----
340 // Delay lines interpolated using Thiran allpass interpolation.
341 //

```

```

342 // #### Usage
343 //
344 // '''
345 // _ : fdelay[N]a(n, d) : _
346 // '''
347 //
348 // (exactly like 'fdelay')
349 //
350 // Where:
351 //
352 // * 'N=1,2,3, or 4' is the order of the Thiran interpolation filter (constant numerical
    expression),
353 // and the delay argument is at least 'N-1/2'. First-order: 'd' at least 0.5, second-order
    : 'd' at least 1.5,
354 // third-order: 'd' at least 2.5, fourth-order: 'd' at least 3.5.
355 // * 'n': the max delay length in samples
356 // * 'd': the delay length in samples
357 //
358 // #### Test
359 // '''
360 // de = library("delays.lib");
361 // os = library("oscillators.lib");
362 // fdelay2a_test = os.osc(440) : de.fdelay2a(44100, 22050.5);
363 // '''
364 //
365 // #### Note
366 //
367 // The interpolated delay should not be less than 'N-1/2'.
368 // (The allpass delay ranges from 'N-1/2' to 'N+1/2').
369 // This constraint can be alleviated by altering the code,
370 // but be aware that allpass filters approach zero delay
371 // by means of pole-zero cancellations.
372 //
373 // Delay arguments too small will produce an UNSTABLE allpass!
374 //
375 // Because allpass interpolation is recursive, it is not as robust
376 // as Lagrange interpolation under time-varying conditions
377 // (you may hear clicks when changing the delay rapidly).
378 //
379 //-----
380 declare fdelay1a author "Julius O. Smith III";
381
382 fdelay1a(n,d,x) = delay(n,id,x) : fi.tf1(eta,1,eta)
383 with {
384   o = 0.49999; // offset to make life easy for allpass
385   dmo = d - o; // assumed nonnegative
386   id = int(dmo);
387   fd = o + ma.frac(dmo);
388   eta = (1-fd)/(1+fd); // allpass coefficient
389 };
390
391 declare fdelay2a author "Julius O. Smith III";
392 fdelay2a(n,d,x) = delay(n,id,x) : fi.tf2(a2,a1,1,a1,a2)
393 with {
394   o = 1.49999;
395   dmo = d - o; // delay range is [order-1/2, order+1/2]
396   id = int(dmo);
397   fd = o + ma.frac(dmo);
398   a1o2 = (2-fd)/(1+fd); // share some terms (the compiler does this anyway)
399   a1 = 2*a1o2;
400   a2 = a1o2*(1-fd)/(2+fd);
401 };
402
403 declare fdelay3a author "Julius O. Smith III";
404 fdelay3a(n,d,x) = delay(n,id,x) : fi.iir((a3,a2,a1,1),(a1,a2,a3))
405 with {
406   o = 2.49999;
407   dmo = d - o;

```

```

408     id = int(dmo);
409     fd = o + ma.frac(dmo);
410     a1o3 = (3-fd)/(1+fd);
411     a2o3 = a1o3*(2-fd)/(2+fd);
412     a1 = 3*a1o3;
413     a2 = 3*a2o3;
414     a3 = a2o3*(1-fd)/(3+fd);
415 };
416
417 declare fdelay4a author "Julius O. Smith III";
418 fdelay4a(n,d,x) = delay(n,id,x) : fi.iir((a4,a3,a2,a1,1),(a1,a2,a3,a4))
419 with {
420     o = 3.49999;
421     dmo = d - o;
422     id = int(dmo);
423     fd = o + ma.frac(dmo);
424     a1o4 = (4-fd)/(1+fd);
425     a2o6 = a1o4*(3-fd)/(2+fd);
426     a3o4 = a2o6*(2-fd)/(3+fd);
427     a1 = 4*a1o4;
428     a2 = 6*a2o6;
429     a3 = 4*a3o4;
430     a4 = a3o4*(1-fd)/(4+fd);
431 };
432
433 //=====Others=====
434 //=====
435
436 //-----'(de.)multiTapSincDelay'-----
437 //
438 // Variable delay line using multi-tap sinc interpolation.
439 //
440 // This function implements a continuously variable delay line by superposing (2K+2)
441 //   auxiliary delayed signals
442 // whose positions and gains are determined by a sinc-based interpolation method. It extends
443 //   the traditional
444 // crossfade delay technique to significantly reduce spectral coloration artifacts, which
445 //   are problematic in
446 // applications like Wave Field Synthesis (WFS) and auralization.
447 //
448 // Operation:
449 //
450 // - If tau1 and tau2 are very close (|tau2 - tau1| 0), a simple fixed fractional delay is
451 //   applied
452 // - Otherwise, a variable delay is synthesized by:
453 //
454 //   - Computing (2K+2) taps symmetrically distributed around tau1 and tau2
455 //   - Applying sinc-based weighting to each tap, based on its offset from the target
456 //   - Summing all the weighted taps to produce the output
457 //
458 // Features:
459 //
460 // - Smooth delay variation without introducing Doppler pitch shifts
461 // - Significant reduction of comb-filter coloration compared to classical crossfading
462 // - Switching between fixed and variable delay modes to ensure stability
463 //
464 // Usage
465 //
466 // _ : multiTapSincDelay(K, MaxDelay, tau1, tau2, alpha) : _
467 //
468 // Where:
469 //
470 // * 'K (integer)': number of auxiliary tap pairs (a constant numerical expression). Total
471 //   number of taps = 2*K + 2
472 // * 'MaxDelay': maximum allowable delay in samples (buffer size)

```

```

470 // * 'tau1': initial delay in samples (can be fractional)
471 // * 'tau2': target delay in samples (can be fractional)
472 // * 'alpha': interpolation factor between tau1 and tau2 (in [0,1] with 0 = tau1, 1 = tau2)
473 //
474 // #### Test
475 // '''
476 // de = library("delays.lib");
477 // os = library("oscillators.lib");
478 // multiTapSincDelay_test = os.osc(440) : de.multiTapSincDelay(2, 4096, 1024.0, 1536.0, 0.5)
479 // ;
480 //
481 // #### References
482 //
483 // T. Carpentier, "Implementation of a continuously variable delay line by crossfading
484 // between several tap delays", 2024: <https://hal.science/hal-04646939>
485 //-----
486 multiTapSincDelay(K, MaxDelay, tau1, tau2, alpha, input) = output with {
487
488     // Total number of taps used in the sum (2K + 2).
489     numTaps = 2 * K + 2;
490
491     // Normalized Sinc function: sinc(x) = sin(pi*x)/(pi*x)
492     // Handles the case where x = 0 to avoid division by zero and return 1.
493     sinc(x) = ba.if(abs(x) < ma.EPSILON, 1, sin(ma.PI * x) / (ma.PI * x));
494
495     // Computes the difference between two delays: delta = tau2 - tau1
496     delta(t1, t2) = t2 - t1;
497
498     // Computes the target interpolated delay: tau = (1-alpha)*tau1 + alpha*tau2
499     // This is the center of the sinc function for the hk gains.
500     tau(t1, t2, a) = (1 - a) * t1 + a * t2;
501
502     // Computes the position (delay in samples) of the i-th tap (tk),
503     // according to equation (17) of the article.
504     tk(i, t1, t2) = ba.if(i <= K,
505         t1 - (K - i) * delta(t1, t2),
506         t2 + (i - K - 1) * delta(t1, t2));
507
508     // Computes the gain (hk) of the i-th tap, according to equation (19) of the paper.
509     // hk = sinc((tk - tau) / delta)
510     hk(i, t1, t2, a) = sinc(offset) with {
511         d = delta(t1, t2);
512         denom = ba.if(abs(d) < ma.EPSILON, 1, d);
513         offset = (tk(i, t1, t2) - tau(t1, t2, a)) / denom;
514     };
515
516     // Computes the sum of delayed signals weighted by their gains.
517     // This is the core of the variable multi-tap effect.
518     tapSum(sig, t1, t2, a) =
519         // Iterates from i = 0 to numTaps-1
520         sum(i, numTaps,
521             // For each tap 'i':
522             // 1. Delays the signal 'sig' by tk(i) samples (fractional)
523             // using an internal delay line (fdelay).
524             // 2. Multiplies the delayed signal by the gain hk(i).
525             sig : fdelay(MaxDelay, tk(i, t1, t2)) * hk(i, t1, t2, a)
526         );
527
528     // Simple function to apply a fixed delay (potentially fractional).
529     // Used when tau1 and tau2 are very close.
530     fixedDelay(x, t) = x : fdelay(MaxDelay, t);
531
532     // Determines if delays tau1 and tau2 are close enough to be
533     // considered fixed (to avoid unstable calculations when delta = 0).
534     // isFixed equals 1 if delta = 0, otherwise 0.
535     isFixed = abs(delta(tau1, tau2)) < ma.EPSILON;

```

```

536 // Computes the output corresponding to the fixed delay (used if isFixed=1).
537 fd = fixedDelay(input, tau1);
538
539 // Computes the output corresponding to the variable multi-tap delay (used if isFixed=0).
540 mt = tapSum(input, tau1, tau2, alpha);
541
542 // Selects the final output based on isFixed.
543 output = ba.if(isFixed, fd, mt);
544 };
545
546
547 ///////////////////////////////////////////////////Deprecated Functions//////////////////////////////////////
548 // This section implements functions that used to be in music.lib but that are now
549 // considered as "deprecated".
550 ///////////////////////////////////////////////////
551
552 delay1s(d) = delay(65536,d);
553 delay2s(d) = delay(131072,d);
554 delay5s(d) = delay(262144,d);
555 delay10s(d) = delay(524288,d);
556 delay21s(d) = delay(1048576,d);
557 delay43s(d) = delay(2097152,d);
558
559 fdelay1s(d) = fdelay(65536,d);
560 fdelay2s(d) = fdelay(131072,d);
561 fdelay5s(d) = fdelay(262144,d);
562 fdelay10s(d) = fdelay(524288,d);
563 fdelay21s(d) = fdelay(1048576,d);
564 fdelay43s(d) = fdelay(2097152,d);

```

Listing 4: maths.lib

```

1 //##### maths.lib #####
2 // Maths library. Its official prefix is 'ma'.
3 //
4 // This library provides mathematical functions and utilities for numerical
5 // computations in Faust. It includes trigonometric, exponential, logarithmic, and
6 // statistical functions, constants, and complex-number
7 // operations used throughout Faust DSP and control code.
8 //
9 // The Maths library is organized into 1 section:
10 //
11 // * [Functions Reference] (#functions-reference)
12 //
13 // Some functions are implemented as Faust foreign functions of 'math.h' functions
14 // that are not part of Faust's primitives. Defines also various constants and several
15 // utilities.
16 //
17 // ### References
18 //
19 // * <https://github.com/grame-cncm/faustlibraries/blob/master/maths.lib>
20 //#####
21
22 // ## History
23 // * 06/13/2016 [RM] normalizing and integrating to new libraries
24 // * 07/08/2015 [YO] documentation comments
25 // * 20/06/2014 [SL] added FTZ function
26 // * 22/06/2013 [YO] added float|double|quad variants of some foreign functions
27 // * 28/06/2005 [YO] postfix functions with 'f' to force float version instead of double
28 // * 28/06/2005 [YO] removed 'modf' because it requires a pointer as argument
29
30 /*****
31 *****/
32 FAUST library file

```

```

33 Copyright (C) 2003-2016 GRAME, Centre National de Creation Musicale
34 -----
35 This program is free software; you can redistribute it and/or modify
36 it under the terms of the GNU Lesser General Public License as
37 published by the Free Software Foundation; either version 2.1 of the
38 License, or (at your option) any later version.
39
40 This program is distributed in the hope that it will be useful,
41 but WITHOUT ANY WARRANTY; without even the implied warranty of
42 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
43 GNU Lesser General Public License for more details.
44
45 You should have received a copy of the GNU Lesser General Public
46 License along with the GNU C Library; if not, write to the Free
47 Software Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA
48 02111-1307 USA.
49
50 EXCEPTION TO THE LGPL LICENSE : As a special exception, you may create a
51 larger FAUST program which directly or indirectly imports this library
52 file and still distribute the compiled code generated by the FAUST
53 compiler, or a modified version of this compiled code, under your own
54 copyright and license. This EXCEPTION TO THE LGPL LICENSE explicitly
55 grants you the right to freely choose the license for the resulting
56 compiled code. In particular the resulting compiled code has no obligation
57 to be LGPL or GPL. For example you are free to choose a commercial or
58 closed source license or any other license if you decide so.
59 *****/
60 *****/
61
62 // This library contains platform specific constants
63 ba = library("basics.lib");
64 pl = library("platform.lib");
65 ma = library("maths.lib"); // for compatible copy/paste out of this file
66
67 declare name "Faust Math Library";
68 declare version "2.9.0";
69 declare author "GRAME";
70 declare copyright "GRAME";
71 declare license "LGPL with exception";
72
73 //=====Functions Reference=====
74 //=====
75
76
77 //-----'(ma.)SR'-----
78 // Current sampling rate given at init time. Constant during program execution.
79 //
80 // #### Usage
81 //
82 // '
83 // SR : -
84 // '
85 //
86 // Where:
87 //
88 // * 'SR': initialization-time sampling rate constant
89 //
90 // #### Test
91 // '
92 // ma = library("maths.lib");
93 // SR_test = ma.SR;
94 // '
95 //-----
96 SR = pl.SR;
97
98 //-----'(ma.)T'-----
99 // Current sample duration in seconds computed from the sampling rate given at init time.
    Constant during program execution.

```

```

100 //
101 // ##### Usage
102 //
103 // '''
104 // T : _
105 // '''
106 //
107 // Where:
108 //
109 // * 'T': sample duration ('1/SR') constant
110 //
111 // ##### Test
112 // '''
113 // ma = library("maths.lib");
114 // T_test = ma.T;
115 // '''
116 //-----
117 T = 1.0 / SR;
118
119 //-----'(ma.)BS'-----
120 // Current block-size. Can change during the execution at each block.
121 //
122 // ##### Usage
123 //
124 // '''
125 // BS : _
126 // '''
127 //
128 // Where:
129 //
130 // * 'BS': current processing block size
131 //
132 // ##### Test
133 // '''
134 // ma = library("maths.lib");
135 // BS_test = ma.BS;
136 // '''
137 //-----
138 BS = pl.BS;
139
140
141 //-----'(ma.)PI'-----
142 // Constant PI in double precision.
143 //
144 // ##### Usage
145 //
146 // '''
147 // PI : _
148 // '''
149 //
150 // Where:
151 //
152 // * 'PI': double-precision constant
153 //
154 // ##### Test
155 // '''
156 // ma = library("maths.lib");
157 // PI_test = ma.PI;
158 // '''
159 //-----
160 PI = 3.14159265358979323846;
161
162
163 //-----'(ma.)deg2rad'-----
164 // Convert degrees to radians.
165 //
166 // ##### Usage
167 //

```

```

168 // '''
169 // 45. : deg2rad
170 // '''
171 //
172 // Where:
173 //
174 // * input: angle in degrees to convert
175 //
176 // #### Test
177 // '''
178 // ma = library("maths.lib");
179 // deg2rad_test = 45.0 : ma.deg2rad;
180 // '''
181 //-----
182 deg2rad = _ * PI / 180.;
183
184
185 //-----'(ma.)rad2deg'-----
186 // Convert radians to degrees.
187 //
188 // #### Usage
189 //
190 // '''
191 // ma.PI : rad2deg
192 // '''
193 //
194 // Where:
195 //
196 // * input: angle in radians to convert
197 //
198 // #### Test
199 // '''
200 // ma = library("maths.lib");
201 // rad2deg_test = ma.PI : ma.rad2deg;
202 // '''
203 //-----
204 rad2deg = _ * 180. / PI;
205
206
207 //-----'(ma.)E'-----
208 // Constant e in double precision.
209 //
210 // #### Usage
211 //
212 // '''
213 // E : _
214 // '''
215 //
216 // Where:
217 //
218 // * 'E': double-precision Euler's number constant
219 //
220 // #### Test
221 // '''
222 // ma = library("maths.lib");
223 // E_test = ma.E;
224 // '''
225 //-----
226 E = 2.71828182845904523536;
227
228
229 //-----'(ma.)EPSILON'-----
230 // Constant EPSILON available in simple/double/quad precision,
231 // as defined in the [floating-point standard] (https://en.wikipedia.org/wiki/IEEE\_754)
232 // and [machine epsilon] (https://en.wikipedia.org/wiki/Machine\_epsilon),
233 // that is smallest positive number such that '1.0 + EPSILON != 1.0'.
234 //
235 // #### Usage

```

```

236 //
237 // '''
238 // EPSILON : _
239 // '''
240 //
241 // Where:
242 //
243 // * 'EPSILON': machine epsilon constant for the current floating-point precision
244 //
245 // #### Test
246 // '''
247 // ma = library("maths.lib");
248 // EPSILON_test = ma.EPSILON;
249 // '''
250 //-----
251 singleprecision EPSILON = 1.192092896e-07;
252 doubleprecision EPSILON = 2.2204460492503131e-016;
253 quadprecision EPSILON = 1.084202172485504434007452e-019;
254 fixedpointprecision EPSILON = 2.2204460492503131e-016;
255
256 //-----'(ma.)MIN'-----
257 // Constant MIN available in simple/double/quad precision (minimal positive value).
258 //
259 // #### Usage
260 //
261 // '''
262 //
263 // MIN : _
264 // '''
265 //
266 // Where:
267 //
268 // * 'MIN': minimal positive normalized value for the current precision
269 //
270 // #### Test
271 // '''
272 // ma = library("maths.lib");
273 // MIN_test = ma.MIN * 1e307;
274 // '''
275 //-----
276 singleprecision MIN = 1.175494351e-38;
277 doubleprecision MIN = 2.2250738585072014e-308;
278 quadprecision MIN = 2.2250738585072014e-308;
279 fixedpointprecision MIN = 2.2250738585072014e-308;
280
281 //-----'(ma.)MAX'-----
282 // Constant MAX available in simple/double/quad precision (maximal positive value).
283 //
284 // #### Usage
285 //
286 // '''
287 //
288 // MAX : _
289 // '''
290 //
291 // Where:
292 //
293 // * 'MAX': maximal finite value for the current precision
294 //
295 // #### Test
296 // '''
297 // ma = library("maths.lib");
298 // MAX_test = ma.MAX;
299 // '''
300 //-----
301 singleprecision MAX = 3.402823466e+38;
302 doubleprecision MAX = 1.7976931348623158e+308;
303 quadprecision MAX = 1.7976931348623158e+308;

```

```

304 fixedpointprecision MAX = 1.7976931348623158e+308;
305
306 // Obsolete, kept for compatibility reasons
307 INFINITY = MAX;
308
309 //-----'(ma.)FTZ'-----
310 // Flush to zero: force samples under the "maximum subnormal number"
311 // to be zero. Usually not needed in C++ because the architecture
312 // file take care of this, but can be useful in JavaScript for instance.
313 //
314 // #### Usage
315 //
316 // ' '
317 // _ : FTZ : _
318 // ' '
319 //
320 // Where:
321 //
322 // * 'x': input signal to flush if its magnitude is subnormal
323 //
324 // #### Test
325 // ' '
326 // ma = library("maths.lib");
327 // FTZ_test = ((ma.MIN * 0.5) : ma.FTZ), ((ma.MIN * 1e307) : ma.FTZ);
328 // ' '
329 //
330 // #### References
331 //
332 // <http://docs.oracle.com/cd/E19957-01/806-3568/ncg\_math.html>
333 //-----
334 FTZ(x) = x * (abs(x) > MIN);
335
336
337 //-----'(ma.)copysign'-----
338 // Changes the sign of x (first input) to that of y (second input).
339 //
340 // #### Usage
341 //
342 // ' '
343 // _ , _ : copysign : _
344 // ' '
345 //
346 // Where:
347 //
348 // * 'x': value whose magnitude is preserved
349 // * 'y': value providing the sign
350 //
351 // #### Test
352 // ' '
353 // ma = library("maths.lib");
354 // copysign_test = (-1.0, 2.0) : ma.copysign;
355 // ' '
356 //-----
357 copysign = ffunction(float copysignf|copysign|copysignl (float, float), <math.h>,"");
358
359
360 //-----'(ma.)neg'-----
361 // Invert the sign (-x) of a signal.
362 //
363 // #### Usage
364 //
365 // ' '
366 // _ : neg : _
367 // ' '
368 //
369 // Where:
370 //
371 // * 'x': value to negate

```

```

372 //
373 // #### Test
374 // '''
375 // ma = library("maths.lib");
376 // neg_test = 3.5 : ma.neg;
377 // '''
378 //-----
379 neg(x) = -x;
380
381
382 //-----'(ma.)not'-----
383 // Bitwise 'not' implemented with [xor](https://faustdoc.grame.fr/manual/syntax/#xor-
    primitive) as 'not(x) = x xor -1;'.
384 // So working regardless of the size of the integer, assuming negative numbers in two's
    complement.
385 //
386 // #### Usage
387 //
388 // '''
389 // _ : not : _
390 // '''
391 //
392 // Where:
393 //
394 // * 'x': integer input value
395 //
396 // #### Test
397 // '''
398 // ma = library("maths.lib");
399 // not_test = 5 : ma.not;
400 // '''
401 //-----
402 not(x) = x xor -1;
403
404
405 //-----'(ma.)sub(x,y)'-----
406 // Subtract 'x' and 'y'.
407 //
408 // #### Usage
409 //
410 // '''
411 // _,_ : sub : _
412 // '''
413 //
414 // Where:
415 //
416 // * 'x': first operand
417 // * 'y': second operand
418 //
419 // #### Test
420 // '''
421 // ma = library("maths.lib");
422 // sub_test = (3, 10) : ma.sub;
423 // '''
424 //-----
425 sub(x,y) = y-x;
426
427
428 //-----'(ma.)inv'-----
429 // Compute the inverse (1/x) of the input signal.
430 //
431 // #### Usage
432 //
433 // '''
434 // _ : inv : _
435 // '''
436 //
437 // Where:

```

```

438 //
439 // * 'x': denominator input (non-zero)
440 //
441 // #### Test
442 // '''
443 // ma = library("maths.lib");
444 // inv_test = 4.0 : ma.inv;
445 // '''
446 //-----
447 inv(x) = 1/x;
448
449
450 //-----'(ma.)cbrt'-----
451 // Computes the cube root of of the input signal.
452 //
453 // #### Usage
454 //
455 // '''
456 // _ : cbrt : _
457 // '''
458 //
459 // Where:
460 //
461 // * 'x': value whose cube root is computed
462 //
463 // #### Test
464 // '''
465 // ma = library("maths.lib");
466 // cbrt_test = 8.0 : ma.cbrt;
467 // '''
468 //-----
469 cbrt = ffunction(float cbrtf|cbrt|cbrtl (float), <math.h>,"");
470
471
472 //-----'(ma.)hypot'-----
473 // Computes the euclidian distance of the two input signals
474 // sqrt(x*x+y*y) without undue overflow or underflow.
475 //
476 // #### Usage
477 //
478 // '''
479 // _,_ : hypot : _
480 // '''
481 //
482 // Where:
483 //
484 // * 'x': first operand
485 // * 'y': second operand
486 //
487 // #### Test
488 // '''
489 // ma = library("maths.lib");
490 // hypot_test = (3.0, 4.0) : ma.hypot;
491 // '''
492 //-----
493 hypot = ffunction(float hypotf|hypot|hypotl (float, float), <math.h>,"");
494
495
496 //-----'(ma.)ldexp'-----
497 // Takes two input signals: x and n, and multiplies x by 2 to the power n.
498 //
499 // #### Usage
500 //
501 // '''
502 // _,_ : ldexp : _
503 // '''
504 //
505 // Where:

```

```

506 //
507 // * 'x': significand input
508 // * 'n': exponent (integer) input
509 //
510 // #### Test
511 // '''
512 // ma = library("maths.lib");
513 // ldexp_test = (1.5, 3) : ma.ldexp;
514 // '''
515 //-----
516 ldexp = ffunction(float ldexpf|ldexp|ldexpl (float, int), <math.h>,"");
517
518
519 //-----'(ma.)scalb'-----
520 // Takes two input signals: x and n, and multiplies x by 2 to the power n.
521 //
522 // #### Usage
523 //
524 // '''
525 // _ : scalb : _
526 // '''
527 //
528 // Where:
529 //
530 // * 'x': significand input
531 // * 'n': exponent (integer) input
532 //
533 // #### Test
534 // '''
535 // ma = library("maths.lib");
536 // scalb_test = (2.0, -1) : ma.scalb;
537 // '''
538 //-----
539 scalb = ffunction(float scalbnf|scalbn|scalbnl (float, int), <math.h>,"");
540
541
542 //-----'(ma.)log1p'-----
543 // Computes log(1 + x) without undue loss of accuracy when x is nearly zero.
544 //
545 // #### Usage
546 //
547 // '''
548 // _ : log1p : _
549 // '''
550 //
551 // Where:
552 //
553 // * 'x': offset used in 'log(1 + x)' (must be greater than -1)
554 //
555 // #### Test
556 // '''
557 // ma = library("maths.lib");
558 // log1p_test = 0.5 : ma.log1p;
559 // '''
560 //-----
561 log1p = ffunction(float log1pf|log1p|log1pl (float), <math.h>,"");
562
563
564 //-----'(ma.)logb'-----
565 // Return exponent of the input signal as a floating-point number.
566 //
567 // #### Usage
568 //
569 // '''
570 // _ : logb : _
571 // '''
572 //
573 // Where:

```

```

574 //
575 // * 'x': positive value whose exponent part is returned
576 //
577 // #### Test
578 // '''
579 // ma = library("maths.lib");
580 // logb_test = 8.0 : ma.logb;
581 // '''
582 //-----
583 logb = ffunction(float logbf|logb|logbl (float), <math.h>,"");
584
585
586 //-----'(ma.)ilogb'-----
587 // Return exponent of the input signal as an integer number.
588 //
589 // #### Usage
590 //
591 // '''
592 // _ : ilogb : _
593 // '''
594 //
595 // Where:
596 //
597 // * 'x': positive value whose exponent part is returned
598 //
599 // #### Test
600 // '''
601 // ma = library("maths.lib");
602 // ilogb_test = 8.0 : ma.ilogb;
603 // '''
604 //-----
605 ilogb = ffunction(int ilogbf|ilogb|ilogbl (float), <math.h>,"");
606
607
608 //-----'(ma.)log2'-----
609 // Returns the base 2 logarithm of x.
610 //
611 // #### Usage
612 //
613 // '''
614 // _ : log2 : _
615 // '''
616 //
617 // Where:
618 //
619 // * 'x': positive value whose base-2 logarithm is computed
620 //
621 // #### Test
622 // '''
623 // ma = library("maths.lib");
624 // log2_test = 8.0 : ma.log2;
625 // '''
626 //-----
627 log2(x) = log(x)/log(2.0);
628
629
630 //-----'(ma.)expm1'-----
631 // Return exponent of the input signal minus 1 with better precision.
632 //
633 // #### Usage
634 //
635 // '''
636 // _ : expm1 : _
637 // '''
638 //
639 // Where:
640 //
641 // * 'x': input value used for the 'exp(x) - 1' computation

```

```

642 //
643 // #### Test
644 // '''
645 // ma = library("maths.lib");
646 // expm1_test = 0.5 : ma.expm1;
647 // '''
648 //-----
649 expm1 = ffunction(float expm1f|expm1|expm1l (float), <math.h>,"");
650
651 //-----'(ma.)acosh'-----
652 // Computes the principle value of the inverse hyperbolic cosine
653 // of the input signal.
654 //
655 //
656 // #### Usage
657 //
658 // '''
659 // _ : acosh : _
660 // '''
661 //
662 // Where:
663 //
664 // * 'x': input value (greater than or equal to 1)
665 //
666 // #### Test
667 // '''
668 // ma = library("maths.lib");
669 // acosh_test = 1.5 : ma.acosh;
670 // '''
671 //-----
672 acosh = ffunction(float acoshf|acosh|acoshl (float), <math.h>, "");
673
674 //-----'(ma.)asinh'-----
675 // Computes the inverse hyperbolic sine of the input signal.
676 //
677 //
678 // #### Usage
679 //
680 // '''
681 // _ : asinh : _
682 // '''
683 //
684 // Where:
685 //
686 // * 'x': input value
687 //
688 // #### Test
689 // '''
690 // ma = library("maths.lib");
691 // asinh_test = 0.5 : ma.asinh;
692 // '''
693 //-----
694 asinh = ffunction(float asinhf|asinh|asinh1 (float), <math.h>, "");
695
696 //-----'(ma.)atanh'-----
697 // Computes the inverse hyperbolic tangent of the input signal.
698 //
699 //
700 // #### Usage
701 //
702 // '''
703 // _ : atanh : _
704 // '''
705 //
706 // Where:
707 //
708 // * 'x': input value in (-1, 1)
709 //

```

```

710 // #### Test
711 // '''
712 // ma = library("maths.lib");
713 // atanh_test = 0.5 : ma.atanh;
714 // '''
715 //-----
716 atanh = ffunction(float atanhf|atanh|atanhl (float), <math.h>, "");
717
718
719 //-----'(ma.)sinh'-----
720 // Computes the hyperbolic sine of the input signal.
721 //
722 // #### Usage
723 //
724 // '''
725 // _ : sinh : _
726 // '''
727 //
728 // Where:
729 //
730 // * 'x': input value
731 //
732 // #### Test
733 // '''
734 // ma = library("maths.lib");
735 // sinh_test = 0.5 : ma.sinh;
736 // '''
737 //-----
738 sinh = ffunction(float sinhf|sinh|sinhl (float), <math.h>, "");
739
740
741 //-----'(ma.)cosh'-----
742 // Computes the hyperbolic cosine of the input signal.
743 //
744 // #### Usage
745 //
746 // '''
747 // _ : cosh : _
748 // '''
749 //
750 // Where:
751 //
752 // * 'x': input value
753 //
754 // #### Test
755 // '''
756 // ma = library("maths.lib");
757 // cosh_test = 0.5 : ma.cosh;
758 // '''
759 //-----
760 cosh = ffunction(float coshf|cosh|coshl (float), <math.h>, "");
761
762
763 //-----'(ma.)tanh'-----
764 // Computes the hyperbolic tangent of the input signal.
765 //
766 // #### Usage
767 //
768 // '''
769 // _ : tanh : _
770 // '''
771 //
772 // Where:
773 //
774 // * 'x': input value
775 //
776 // #### Test
777 // '''

```

```

778 // ma = library("maths.lib");
779 // tanh_test = 0.5 : ma.tanh;
780 // '''
781 //-----
782 tanh = ffunction(float tanhf|tanh|tanh1 (float), <math.h>,"");
783
784
785 //-----'(ma.)erf'-----
786 // Computes the error function of the input signal.
787 //
788 // #### Usage
789 //
790 // '''
791 // _ : erf : _
792 // '''
793 //
794 // Where:
795 //
796 // * 'x': input value
797 //
798 // #### Test
799 // '''
800 // ma = library("maths.lib");
801 // erf_test = 0.5 : ma.erf;
802 // '''
803 //-----
804 erf = ffunction(float erff|erf|erfl(float), <math.h>,"");
805
806
807 //-----'(ma.)erfc'-----
808 // Computes the complementary error function of the input signal.
809 //
810 // #### Usage
811 //
812 // '''
813 // _ : erfc : _
814 // '''
815 //
816 // Where:
817 //
818 // * 'x': input value
819 //
820 // #### Test
821 // '''
822 // ma = library("maths.lib");
823 // erfc_test = 0.5 : ma.erfc;
824 // '''
825 //-----
826 erfc = ffunction(float erfcf|erfc|erfcl(float), <math.h>,"");
827
828
829 //-----'(ma.)gamma'-----
830 // Computes the gamma function of the input signal.
831 //
832 // #### Usage
833 //
834 // '''
835 // _ : gamma : _
836 // '''
837 //
838 // Where:
839 //
840 // * 'x': positive input value
841 //
842 // #### Test
843 // '''
844 // ma = library("maths.lib");
845 // gamma_test = 3.0 : ma.gamma;

```

```

846 // '''
847 //-----'(ma.)lgamma'-----
848 gamma = ffunction(float tgammaf|tgamma|tgamma(float), <math.h>,"");
849
850
851 //-----'(ma.)lgamma'-----
852 // Calculates the natural logarithm of the absolute value of
853 // the gamma function of the input signal.
854 //
855 // #### Usage
856 //
857 // '''
858 // _ : lgamma : _
859 // '''
860 //
861 // Where:
862 //
863 // * 'x': positive input value
864 //
865 // #### Test
866 // '''
867 // ma = library("maths.lib");
868 // lgamma_test = 3.0 : ma.lgamma;
869 // '''
870 //-----
871 lgamma = ffunction(float lgammaf|lgamma|lgamma(float), <math.h>,"");
872
873
874 //-----'(ma.)J0'-----
875 // Computes the Bessel function of the first kind of order 0
876 // of the input signal.
877 //
878 // #### Usage
879 //
880 // '''
881 // _ : J0 : _
882 // '''
883 //
884 // Where:
885 //
886 // * 'x': input value
887 //
888 // #### Test
889 // '''
890 // ma = library("maths.lib");
891 // J0_test = 1.0 : ma.J0;
892 // '''
893 //-----
894 J0 = ffunction(float j0(float), <math.h>,"");
895
896
897 //-----'(ma.)J1'-----
898 // Computes the Bessel function of the first kind of order 1
899 // of the input signal.
900 //
901 // #### Usage
902 //
903 // '''
904 // _ : J1 : _
905 // '''
906 //
907 // Where:
908 //
909 // * 'x': input value
910 //
911 // #### Test
912 // '''
913 // ma = library("maths.lib");

```

```

914 // J1_test = 1.0 : ma.J1;
915 // '''
916 //-----
917 J1 = ffunction(float j1(float), <math.h>,"");
918
919
920 //-----'(ma.)Jn'-----
921 // Computes the Bessel function of the first kind of order n
922 // (first input signal) of the second input signal.
923 //
924 // #### Usage
925 //
926 // '''
927 // _ : Jn : _
928 // '''
929 //
930 // Where:
931 //
932 // * 'n': integer order
933 // * 'x': input value
934 //
935 // #### Test
936 // '''
937 // ma = library("maths.lib");
938 // Jn_test = (2, 1.0) : ma.Jn;
939 // '''
940 //-----
941 Jn = ffunction(float jn(int, float), <math.h>,"");
942
943
944 //-----'(ma.)Y0'-----
945 // Computes the linearly independent Bessel function of the second kind
946 // of order 0 of the input signal.
947 //
948 // #### Usage
949 //
950 // '''
951 // _ : Y0 : _
952 // '''
953 //
954 // Where:
955 //
956 // * 'x': positive input value
957 //
958 // #### Test
959 // '''
960 // ma = library("maths.lib");
961 // Y0_test = 1.0 : ma.Y0;
962 // '''
963 //-----
964 Y0 = ffunction(float y0(float), <math.h>,"");
965
966
967 //-----'(ma.)Y1'-----
968 // Computes the linearly independent Bessel function of the second kind
969 // of order 1 of the input signal.
970 //
971 // #### Usage
972 //
973 // '''
974 // _ : Y0 : _
975 // '''
976 //
977 // Where:
978 //
979 // * 'x': positive input value
980 //
981 // #### Test

```

```

982 // '''
983 // ma = library("maths.lib");
984 // Yl_test = 1.0 : ma.Y1;
985 // '''
986 //-----
987 Y1 = ffunction(float y1(float), <math.h>,"");
988
989
990 //-----'(ma.)Yn'-----
991 // Computes the linearly independent Bessel function of the second kind
992 // of order n (first input signal) of the second input signal.
993 //
994 // #### Usage
995 //
996 // '''
997 // _ : _ : Yn : _
998 // '''
999 //
1000 // Where:
1001 //
1002 // * 'n': integer order
1003 // * 'x': positive input value
1004 //
1005 // #### Test
1006 // '''
1007 // ma = library("maths.lib");
1008 // Yn_test = (2, 1.0) : ma.Yn;
1009 // '''
1010 //-----
1011 Yn = ffunction(float yn(int, float), <math.h>,"");
1012
1013
1014 //-----'(ma.)fabs', '(ma.)fmax', '(ma.)fmin
1015 //-----
1016 // Just for compatibility...
1017 //
1018 // '''
1019 // fabs = abs
1020 // fmax = max
1021 // fmin = min
1022 // '''
1023 //-----
1024 fabs = abs;
1025 fmax = max;
1026 fmin = min;
1027
1028 //-----'(ma.)np2'-----
1029 // Gives the next power of 2 of x.
1030 //
1031 // #### Usage
1032 //
1033 // '''
1034 // np2(n) : _
1035 // '''
1036 //
1037 // Where:
1038 //
1039 // * 'n': an integer
1040 //
1041 // #### Test
1042 // '''
1043 // ma = library("maths.lib");
1044 // np2_test = 5 : ma.np2;
1045 // '''
1046 //-----
1047 np2 = -(1) <: >>(1)|_ <: >>(2)|_ <: >>(4)|_ <: >>(8)|_ <: >>(16)|_ : +(1);
1048

```

```

1049 //-----'(ma.)frac'-----
1050 // Gives the fractional part of n.
1051 //
1052 // #### Usage
1053 //
1054 // '''
1055 // frac(n) : _
1056 // '''
1057 //
1058 // Where:
1059 //
1060 // * 'n': a decimal number
1061 //
1062 // #### Test
1063 // '''
1064 // ma = library("maths.lib");
1065 // frac_test = 3.75 : ma.frac;
1066 // '''
1067 //-----
1068 frac(n) = n - floor(n);
1069 decimal = frac;
1070
1071 // NOTE: decimal does the same thing as frac but using floor instead. JDS uses frac a lot
1072 // in filters.lib so we decided to keep that one... decimal is declared though for
1073 // backward compatibility.
1074 // decimal(n) = n - floor(n);
1075
1076
1077 //-----'(ma.)modulo'-----
1078 // Modulus operation using the '(x%y+y)%y' formula to ensures the result is always non-
1079 // negative, even if 'x' is negative.
1080 //
1081 // #### Usage
1082 //
1083 // '''
1084 // modulo(x,y) : _
1085 // '''
1086 //
1087 // Where:
1088 //
1089 // * 'x': the numerator
1090 // * 'y': the denominator
1091 //
1092 // #### Test
1093 // '''
1094 // ma = library("maths.lib");
1095 // modulo_test = (-3, 4) : ma.modulo;
1096 // '''
1097 //-----
1098 modulo(x,y) = (x % y + y) % y;
1099
1100 //-----'(ma.)isnan'-----
1101 // Return non-zero if x is a NaN.
1102 //
1103 // #### Usage
1104 //
1105 // '''
1106 // isnan(x)
1107 // _ : isnan : _
1108 // '''
1109 //
1110 // Where:
1111 //
1112 // * 'x': signal to analyse
1113 //
1114 // #### Test
1115 // '''

```

```

1116 // ma = library("maths.lib");
1117 // os = library("oscillators.lib");
1118 // isnan_test = (os.osc(1) - 2.0) : sqrt : ma.isnan;
1119 // '''
1120 //-----
1121 isnan = ffunction(int isnanf|isnan|isnanl (float),<math.h>,"");
1122
1123
1124 //-----'(ma.)isinf'-----
1125 // Return non-zero if x is a positive or negative infinity.
1126 //
1127 // #### Usage
1128 //
1129 // '''
1130 // isinf(x)
1131 // _ : isinf : _
1132 // '''
1133 //
1134 // Where:
1135 //
1136 // * 'x': signal to analyse
1137 //
1138 // #### Test
1139 // '''
1140 // ma = library("maths.lib");
1141 // os = library("oscillators.lib");
1142 // isinf_test = (os.impulse - os.impulse) : log : ma.isinf;
1143 // '''
1144 //-----
1145 isinf = ffunction(int isinff|isinf|isinfl (float),<math.h>,"");
1146
1147 nextafter = ffunction(float nextafter(float, float),<math.h>,"");
1148
1149
1150 //-----'(ma.)chebychev'-----
1151 // Chebychev transformation of order N.
1152 //
1153 // #### Usage
1154 //
1155 // '''
1156 // _ : chebychev(N) : _
1157 // '''
1158 //
1159 // Where:
1160 //
1161 // * 'N': the order of the polynomial, a constant numerical expression
1162 //
1163 // #### Semantics
1164 //
1165 // '''
1166 // T[0](x) = 1,
1167 // T[1](x) = x,
1168 // T[n](x) = 2x*T[n-1](x) - T[n-2](x)
1169 // '''
1170 //
1171 // #### Test
1172 // '''
1173 // ma = library("maths.lib");
1174 // chebychev_test = 0.5 : ma.chebychev(3);
1175 // '''
1176 //
1177 // #### References
1178 //
1179 // * <http://en.wikipedia.org/wiki/Chebyshev\_polynomial>
1180 //-----
1181 chebychev(0,x) = 1;
1182 chebychev(1,x) = x;
1183 chebychev(n,x) = 2*x*chebychev(n-1, x) - chebychev(n-2, x);

```

```

1184
1185
1186 //-----'(ma.)chebychevpoly'-----
1187 // Linear combination of the first Chebyshev polynomials.
1188 //
1189 // #### Usage
1190 //
1191 // '''
1192 // _ : chebychevpoly((c0,c1,...,cn)) : _
1193 // '''
1194 //
1195 // Where:
1196 //
1197 // * 'cn': the different Chebyshev polynomials such that:
1198 // chebychevpoly((c0,c1,...,cn)) = Sum of chebychev(i)*ci
1199 //
1200 // #### Test
1201 // '''
1202 // ma = library("maths.lib");
1203 // chebychevpoly_test = 0.5 : ma.chebychevpoly((1, 0, 1));
1204 // '''
1205 //
1206 // #### References
1207 //
1208 // * <http://www.csounds.com/manual/html/chebyshevpoly.html>
1209 //-----
1210 chebychevpoly(lcoef) = _ <: L(0,lcoef) :> _
1211 with {
1212     L(n,(c,cs)) = chebychev(n)*c, L(n+1,cs);
1213     L(n,c)      = chebychev(n)*c;
1214 };
1215
1216
1217 //-----'(ma.)diffn'-----
1218 // Negated first-order difference.
1219 //
1220 // #### Usage
1221 //
1222 // '''
1223 // _ : diffn : _
1224 // '''
1225 //
1226 // Where:
1227 //
1228 // * 'x': input signal
1229 //
1230 // #### Test
1231 // '''
1232 // ma = library("maths.lib");
1233 // os = library("oscillators.lib");
1234 // diffn_test = os.osc(440) : ma.diffn;
1235 // '''
1236 //-----
1237 diffn(x) = x' - x; // negated first-order difference
1238
1239
1240 //-----'(ma.)signum'-----
1241 // The signum function signum(x) is defined as
1242 // -1 for x<0, 0 for x==0, and 1 for x>0.
1243 //
1244 // #### Usage
1245 //
1246 // '''
1247 // _ : signum : _
1248 // '''
1249 //
1250 // Where:
1251 //

```

```

1252 // * 'x': input value
1253 //
1254 // #### Test
1255 // '''
1256 // ma = library("maths.lib");
1257 // signum_test = (-5.0) : ma.signum;
1258 // '''
1259 //-----
1260 signum(x) = (x>0)-(x<0);
1261
1262
1263 //-----'(ma.)nextpow2'-----
1264 // The nextpow2(x) returns the lowest integer m such that
1265 // 2^m >= x.
1266 //
1267 // #### Usage
1268 //
1269 // '''
1270 // 2^nextpow2(n) : _
1271 // '''
1272 // Useful for allocating delay lines, e.g.,
1273 // '''
1274 // delay(2^nextpow2(maxDelayNeeded), currentDelay);
1275 // '''
1276 //
1277 // Where:
1278 //
1279 // * 'n': positive value whose next power-of-two exponent is computed
1280 //
1281 // #### Test
1282 // '''
1283 // ma = library("maths.lib");
1284 // nextpow2_test = 10.0 : ma.nextpow2;
1285 // '''
1286 //-----
1287 nextpow2(x) = ceil(log(x)/log(2.0));
1288
1289
1290 //-----'(ma.)zc'-----
1291 // Indicator function for zero-crossing: it returns 1 if a zero-crossing
1292 // occurs, 0 otherwise.
1293 //
1294 // #### Usage
1295 //
1296 // '''
1297 // _ : zc : _
1298 // '''
1299 //
1300 // Where:
1301 //
1302 // * 'x': input signal to monitor for zero crossings
1303 //
1304 // #### Test
1305 // '''
1306 // ma = library("maths.lib");
1307 // os = library("oscillators.lib");
1308 // zc_test = os.osc(440) : ma.zc;
1309 // '''
1310 //-----
1311 zc = ba.tAndH(!=0) <: *(mem) < 0;
1312
1313 //-----'(ma.)unwrap' -----
1314 // Unwrap the input signal so that successive output values never differ
1315 // by more than 'pi', switching to a 2*pi-complementary value when needed.
1316 //
1317 // #### Usage
1318 //
1319 // '''

```

```

1320 // _ : unwrap(pi) : _
1321 // '
1322 //
1323 // Where:
1324 //
1325 // * 'pi': maximum discontinuity between the output values (typically 'ma.PI')
1326 //
1327 // #### Test
1328 // '
1329 // ma = library("maths.lib");
1330 // os = library("oscillators.lib");
1331 // unwrap_test = os.oscrc(100) : ma.unwrap(ma.PI);
1332 // '
1333 //
1334 // #### Example test program
1335 //
1336 // '
1337 // process = 0 - os.oscrc(1000) // the true phase is either -PI or +PI
1338 // : an.resonator(1,1000) : !, _ // oscillates between -PI and +PI
1339 // : ma.unwrap(ma.PI); // oscillates near +PI
1340 // '
1341 //
1342 //-----
1343 unwrap(pi,x) = (_ <: +(dz)) ~ _ with {
1344   dz(z) = wrap(x - z, 2*pi) <: ba.ifNc(1, >(pi), -(2*pi));
1345   wrap(x,d) = x - d * floor(x / d);
1346 };
1347
1348 //-----'(ma.)primes'-----
1349 // Return the n-th prime using a waveform primitive. Note that primes(0) is 2,
1350 // primes(1) is 3, and so on. The waveform is length 2048, so the largest
1351 // precomputed prime is primes(2047) which is 17863.
1352 //
1353 // #### Usage
1354 //
1355 // '
1356 // _ : primes : _
1357 // '
1358 //
1359 // Where:
1360 //
1361 // * 'x': index of the prime number sequence (0-based).
1362 //
1363 // #### Test
1364 // '
1365 // ma = library("maths.lib");
1366 // primes_test = 10 : ma.primes;
1367 // '
1368 //-----
1369 primes(x) = rdttable(waveform{
1370 2,3,5,7,11,13,17,19,23,29,
1371 31,37,41,43,47,53,59,61,67,71,
1372 73,79,83,89,97,101,103,107,109,113,
1373 127,131,137,139,149,151,157,163,167,173,
1374 179,181,191,193,197,199,211,223,227,229,
1375 233,239,241,251,257,263,269,271,277,281,
1376 283,293,307,311,313,317,331,337,347,349,
1377 353,359,367,373,379,383,389,397,401,409,
1378 419,421,431,433,439,443,449,457,461,463,
1379 467,479,487,491,499,503,509,521,523,541,
1380 547,557,563,569,571,577,587,593,599,601,
1381 607,613,617,619,631,641,643,647,653,659,
1382 661,673,677,683,691,701,709,719,727,733,
1383 739,743,751,757,761,769,773,787,797,809,
1384 811,821,823,827,829,839,853,857,859,863,
1385 877,881,883,887,907,911,919,929,937,941,
1386 947,953,967,971,977,983,991,997,1009,1013,
1387 1019,1021,1031,1033,1039,1049,1051,1061,1063,1069,

```

1388	1087,1091,1093,1097,1103,1109,1117,1123,1129,1151,
1389	1153,1163,1171,1181,1187,1193,1201,1213,1217,1223,
1390	1229,1231,1237,1249,1259,1277,1279,1283,1289,1291,
1391	1297,1301,1303,1307,1319,1321,1327,1361,1367,1373,
1392	1381,1399,1409,1423,1427,1429,1433,1439,1447,1451,
1393	1453,1459,1471,1481,1483,1487,1489,1493,1499,1511,
1394	1523,1531,1543,1549,1553,1559,1567,1571,1579,1583,
1395	1597,1601,1607,1609,1613,1619,1621,1627,1637,1657,
1396	1663,1667,1669,1693,1697,1699,1709,1721,1723,1733,
1397	1741,1747,1753,1759,1777,1783,1787,1789,1801,1811,
1398	1823,1831,1847,1861,1867,1871,1873,1877,1879,1889,
1399	1901,1907,1913,1931,1933,1949,1951,1973,1979,1987,
1400	1993,1997,1999,2003,2011,2017,2027,2029,2039,2053,
1401	2063,2069,2081,2083,2087,2089,2099,2111,2113,2129,
1402	2131,2137,2141,2143,2153,2161,2179,2203,2207,2213,
1403	2221,2237,2239,2243,2251,2267,2269,2273,2281,2287,
1404	2293,2297,2309,2311,2333,2339,2341,2347,2351,2357,
1405	2371,2377,2381,2383,2389,2393,2399,2411,2417,2423,
1406	2437,2441,2447,2459,2467,2473,2477,2503,2521,2531,
1407	2539,2543,2549,2551,2557,2579,2591,2593,2609,2617,
1408	2621,2633,2647,2657,2659,2663,2671,2677,2683,2687,
1409	2689,2693,2699,2707,2711,2713,2719,2729,2731,2741,
1410	2749,2753,2767,2777,2789,2791,2797,2801,2803,2819,
1411	2833,2837,2843,2851,2857,2861,2879,2887,2897,2903,
1412	2909,2917,2927,2939,2953,2957,2963,2969,2971,2999,
1413	3001,3011,3019,3023,3037,3041,3049,3061,3067,3079,
1414	3083,3089,3109,3119,3121,3137,3163,3167,3169,3181,
1415	3187,3191,3203,3209,3217,3221,3229,3251,3253,3257,
1416	3259,3271,3299,3301,3307,3313,3319,3323,3329,3331,
1417	3343,3347,3359,3361,3371,3373,3389,3391,3407,3413,
1418	3433,3449,3457,3461,3463,3467,3469,3491,3499,3511,
1419	3517,3527,3529,3533,3539,3541,3547,3557,3559,3571,
1420	3581,3583,3593,3607,3613,3617,3623,3631,3637,3643,
1421	3659,3671,3673,3677,3691,3697,3701,3709,3719,3727,
1422	3733,3739,3761,3767,3769,3779,3793,3797,3803,3821,
1423	3823,3833,3847,3851,3853,3863,3877,3881,3889,3907,
1424	3911,3917,3919,3923,3929,3931,3943,3947,3967,3989,
1425	4001,4003,4007,4013,4019,4021,4027,4049,4051,4057,
1426	4073,4079,4091,4093,4099,4111,4127,4129,4133,4139,
1427	4153,4157,4159,4177,4201,4211,4217,4219,4229,4231,
1428	4241,4243,4253,4259,4261,4271,4273,4283,4289,4297,
1429	4327,4337,4339,4349,4357,4363,4373,4391,4397,4409,
1430	4421,4423,4441,4447,4451,4457,4463,4481,4483,4493,
1431	4507,4513,4517,4519,4523,4547,4549,4561,4567,4583,
1432	4591,4597,4603,4621,4637,4639,4643,4649,4651,4657,
1433	4663,4673,4679,4691,4703,4721,4723,4729,4733,4751,
1434	4759,4783,4787,4789,4793,4799,4801,4813,4817,4831,
1435	4861,4871,4877,4889,4903,4909,4919,4931,4933,4937,
1436	4943,4951,4957,4967,4969,4973,4987,4993,4999,5003,
1437	5009,5011,5021,5023,5039,5051,5059,5077,5081,5087,
1438	5099,5101,5107,5113,5119,5147,5153,5167,5171,5179,
1439	5189,5197,5209,5227,5231,5233,5237,5261,5273,5279,
1440	5281,5297,5303,5309,5323,5333,5347,5351,5381,5387,
1441	5393,5399,5407,5413,5417,5419,5431,5437,5441,5443,
1442	5449,5471,5477,5479,5483,5501,5503,5507,5519,5521,
1443	5527,5531,5557,5563,5569,5573,5581,5591,5623,5639,
1444	5641,5647,5651,5653,5657,5659,5669,5683,5689,5693,
1445	5701,5711,5717,5737,5741,5743,5749,5779,5783,5791,
1446	5801,5807,5813,5821,5827,5839,5843,5849,5851,5857,
1447	5861,5867,5869,5879,5881,5897,5903,5923,5927,5939,
1448	5953,5981,5987,6007,6011,6029,6037,6043,6047,6053,
1449	6067,6073,6079,6089,6091,6101,6113,6121,6131,6133,
1450	6143,6151,6163,6173,6197,6199,6203,6211,6217,6221,
1451	6229,6247,6257,6263,6269,6271,6277,6287,6299,6301,
1452	6311,6317,6323,6329,6337,6343,6353,6359,6361,6367,
1453	6373,6379,6389,6397,6421,6427,6449,6451,6469,6473,
1454	6481,6491,6521,6529,6547,6551,6553,6563,6569,6571,
1455	6577,6581,6599,6607,6619,6637,6653,6659,6661,6673,

1456 6679,6689,6691,6701,6703,6709,6719,6733,6737,6761,
1457 6763,6779,6781,6791,6793,6803,6823,6827,6829,6833,
1458 6841,6857,6863,6869,6871,6883,6899,6907,6911,6917,
1459 6947,6949,6959,6961,6967,6971,6977,6983,6991,6997,
1460 7001,7013,7019,7027,7039,7043,7057,7069,7079,7103,
1461 7109,7121,7127,7129,7151,7159,7177,7187,7193,7207,
1462 7211,7213,7219,7229,7237,7243,7247,7253,7283,7297,
1463 7307,7309,7321,7331,7333,7349,7351,7369,7393,7411,
1464 7417,7433,7451,7457,7459,7477,7481,7487,7489,7499,
1465 7507,7517,7523,7529,7537,7541,7547,7549,7559,7561,
1466 7573,7577,7583,7589,7591,7603,7607,7621,7639,7643,
1467 7649,7669,7673,7681,7687,7691,7699,7703,7717,7723,
1468 7727,7741,7753,7757,7759,7789,7793,7817,7823,7829,
1469 7841,7853,7867,7873,7877,7879,7883,7901,7907,7919,
1470 7927,7933,7937,7949,7951,7963,7993,8009,8011,8017,
1471 8039,8053,8059,8069,8081,8087,8089,8093,8101,8111,
1472 8117,8123,8147,8161,8167,8171,8179,8191,8209,8219,
1473 8221,8231,8233,8237,8243,8263,8269,8273,8287,8291,
1474 8293,8297,8311,8317,8329,8353,8363,8369,8377,8387,
1475 8389,8419,8423,8429,8431,8443,8447,8461,8467,8501,
1476 8513,8521,8527,8537,8539,8543,8563,8573,8581,8597,
1477 8599,8609,8623,8627,8629,8641,8647,8663,8669,8677,
1478 8681,8689,8693,8699,8707,8713,8719,8731,8737,8741,
1479 8747,8753,8761,8779,8783,8803,8807,8819,8821,8831,
1480 8837,8839,8849,8861,8863,8867,8887,8893,8923,8929,
1481 8933,8941,8951,8963,8969,8971,8999,9001,9007,9011,
1482 9013,9029,9041,9043,9049,9059,9067,9091,9103,9109,
1483 9127,9133,9137,9151,9157,9161,9173,9181,9187,9199,
1484 9203,9209,9221,9227,9239,9241,9257,9277,9281,9283,
1485 9293,9311,9319,9323,9337,9341,9343,9349,9371,9377,
1486 9391,9397,9403,9413,9419,9421,9431,9433,9437,9439,
1487 9461,9463,9467,9473,9479,9491,9497,9511,9521,9533,
1488 9539,9547,9551,9587,9601,9613,9619,9623,9629,9631,
1489 9643,9649,9661,9677,9679,9689,9697,9719,9721,9733,
1490 9739,9743,9749,9767,9769,9781,9787,9791,9803,9811,
1491 9817,9829,9833,9839,9851,9857,9859,9871,9883,9887,
1492 9901,9907,9923,9929,9931,9941,9949,9967,9973,10007,
1493 10009,10037,10039,10061,10067,10069,10079,10091,10093,10099,
1494 10103,10111,10133,10139,10141,10151,10159,10163,10169,10177,
1495 10181,10193,10211,10223,10243,10247,10253,10259,10267,10271,
1496 10273,10289,10301,10303,10313,10321,10331,10333,10337,10343,
1497 10357,10369,10391,10399,10427,10429,10433,10453,10457,10459,
1498 10463,10477,10487,10499,10501,10513,10529,10531,10559,10567,
1499 10589,10597,10601,10607,10613,10627,10631,10639,10651,10657,
1500 10663,10667,10687,10691,10709,10711,10723,10729,10733,10739,
1501 10753,10771,10781,10789,10799,10831,10837,10847,10853,10859,
1502 10861,10867,10883,10889,10891,10903,10909,10937,10939,10949,
1503 10957,10973,10979,10987,10993,11003,11027,11047,11057,11059,
1504 11069,11071,11083,11087,11093,11113,11117,11119,11131,11149,
1505 11159,11161,11171,11173,11177,11197,11213,11239,11243,11251,
1506 11257,11261,11273,11279,11287,11299,11311,11317,11321,11329,
1507 11351,11353,11369,11383,11393,11399,11411,11423,11437,11443,
1508 11447,11467,11471,11483,11489,11491,11497,11503,11519,11527,
1509 11549,11551,11579,11587,11593,11597,11617,11621,11633,11657,
1510 11677,11681,11689,11699,11701,11717,11719,11731,11743,11777,
1511 11779,11783,11789,11801,11807,11813,11821,11827,11831,11833,
1512 11839,11863,11867,11887,11897,11903,11909,11923,11927,11933,
1513 11939,11941,11953,11959,11969,11971,11981,11987,12007,12011,
1514 12037,12041,12043,12049,12071,12073,12097,12101,12107,12109,
1515 12113,12119,12143,12149,12157,12161,12163,12197,12203,12211,
1516 12227,12239,12241,12251,12253,12263,12269,12277,12281,12289,
1517 12301,12323,12329,12343,12347,12373,12377,12379,12391,12401,
1518 12409,12413,12421,12433,12437,12451,12457,12473,12479,12487,
1519 12491,12497,12503,12511,12517,12527,12539,12541,12547,12553,
1520 12569,12577,12583,12589,12601,12611,12613,12619,12637,12641,
1521 12647,12653,12659,12671,12689,12697,12703,12713,12721,12739,
1522 12743,12757,12763,12781,12791,12799,12809,12821,12823,12829,
1523 12841,12853,12889,12893,12899,12907,12911,12917,12919,12923,

```

1524 | 12941,12953,12959,12967,12973,12979,12983,13001,13003,13007,
1525 | 13009,13033,13037,13043,13049,13063,13093,13099,13103,13109,
1526 | 13121,13127,13147,13151,13159,13163,13171,13177,13183,13187,
1527 | 13217,13219,13229,13241,13249,13259,13267,13291,13297,13309,
1528 | 13313,13327,13331,13337,13339,13367,13381,13397,13399,13411,
1529 | 13417,13421,13441,13451,13457,13463,13469,13477,13487,13499,
1530 | 13513,13523,13537,13553,13567,13577,13591,13597,13613,13619,
1531 | 13627,13633,13649,13669,13679,13681,13687,13691,13693,13697,
1532 | 13709,13711,13721,13723,13729,13751,13757,13759,13763,13781,
1533 | 13789,13799,13807,13829,13831,13841,13859,13873,13877,13879,
1534 | 13883,13901,13903,13907,13913,13921,13931,13933,13963,13967,
1535 | 13997,13999,14009,14011,14029,14033,14051,14057,14071,14081,
1536 | 14083,14087,14107,14143,14149,14153,14159,14173,14177,14197,
1537 | 14207,14221,14243,14249,14251,14281,14293,14303,14321,14323,
1538 | 14327,14341,14347,14369,14387,14389,14401,14407,14411,14419,
1539 | 14423,14431,14437,14447,14449,14461,14479,14489,14503,14519,
1540 | 14533,14537,14543,14549,14551,14557,14561,14563,14591,14593,
1541 | 14621,14627,14629,14633,14639,14653,14657,14669,14683,14699,
1542 | 14713,14717,14723,14731,14737,14741,14747,14753,14759,14767,
1543 | 14771,14779,14783,14797,14813,14821,14827,14831,14843,14851,
1544 | 14867,14869,14879,14887,14891,14897,14923,14929,14939,14947,
1545 | 14951,14957,14969,14983,15013,15017,15031,15053,15061,15073,
1546 | 15077,15083,15091,15101,15107,15121,15131,15137,15139,15149,
1547 | 15161,15173,15187,15193,15199,15217,15227,15233,15241,15259,
1548 | 15263,15269,15271,15277,15287,15289,15299,15307,15313,15319,
1549 | 15329,15331,15349,15359,15361,15373,15377,15383,15391,15401,
1550 | 15413,15427,15439,15443,15451,15461,15467,15473,15493,15497,
1551 | 15511,15527,15541,15551,15559,15569,15581,15583,15601,15607,
1552 | 15619,15629,15641,15643,15647,15649,15661,15667,15671,15679,
1553 | 15683,15727,15731,15733,15737,15739,15749,15761,15767,15773,
1554 | 15787,15791,15797,15803,15809,15817,15823,15859,15877,15881,
1555 | 15887,15889,15901,15907,15913,15919,15923,15937,15959,15971,
1556 | 15973,15991,16001,16007,16033,16057,16061,16063,16067,16069,
1557 | 16073,16087,16091,16097,16103,16111,16127,16139,16141,16183,
1558 | 16187,16189,16193,16217,16223,16229,16231,16249,16253,16267,
1559 | 16273,16301,16319,16333,16339,16349,16361,16363,16369,16381,
1560 | 16411,16417,16421,16427,16433,16447,16451,16453,16477,16481,
1561 | 16487,16493,16519,16529,16547,16553,16561,16567,16573,16603,
1562 | 16607,16619,16631,16633,16649,16651,16657,16661,16673,16691,
1563 | 16693,16699,16703,16729,16741,16747,16759,16763,16787,16811,
1564 | 16823,16829,16831,16843,16871,16879,16883,16889,16901,16903,
1565 | 16921,16927,16931,16937,16943,16963,16979,16981,16987,16993,
1566 | 17011,17021,17027,17029,17033,17041,17047,17053,17077,17093,
1567 | 17099,17107,17117,17123,17137,17159,17167,17183,17189,17191,
1568 | 17203,17207,17209,17231,17239,17257,17291,17293,17299,17317,
1569 | 17321,17327,17333,17341,17351,17359,17377,17383,17387,17389,
1570 | 17393,17401,17417,17419,17431,17443,17449,17467,17471,17477,
1571 | 17483,17489,17491,17497,17509,17519,17539,17551,17569,17573,
1572 | 17579,17581,17597,17599,17609,17623,17627,17657,17659,17669,
1573 | 17681,17683,17707,17713,17729,17737,17747,17749,17761,17783,
1574 | 17789,17791,17807,17827,17837,17839,17851,17863
1575 | }, x);

```

Listing 5: platform.lib

```

1 //##### platform.lib #####
2 // A library to handle platform specific code in Faust. Its official prefix is 'pl'.
3 //
4 // ### References
5 //
6 // * <https://github.com/grame-cncm/faustlibraries/blob/master/platform.lib>
7
8 //#####
9 // It can be reimplemented to globally change the SR and the tablesizes definitions

```

```

10
11
12 /*****
13 *****/
14 FAUST library file
15 Copyright (C) 2020 GRAME, Centre National de Creation Musicale
16 -----
17 This program is free software; you can redistribute it and/or modify
18 it under the terms of the GNU Lesser General Public License as
19 published by the Free Software Foundation; either version 2.1 of the
20 License, or (at your option) any later version.
21
22 This program is distributed in the hope that it will be useful,
23 but WITHOUT ANY WARRANTY; without even the implied warranty of
24 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
25 GNU Lesser General Public License for more details.
26
27 You should have received a copy of the GNU Lesser General Public
28 License along with the GNU C Library; if not, write to the Free
29 Software Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA
30 02111-1307 USA.
31
32 EXCEPTION TO THE LGPL LICENSE : As a special exception, you may create a
33 larger FAUST program which directly or indirectly imports this library
34 file and still distribute the compiled code generated by the FAUST
35 compiler, or a modified version of this compiled code, under your own
36 copyright and license. This EXCEPTION TO THE LGPL LICENSE explicitly
37 grants you the right to freely choose the license for the resulting
38 compiled code. In particular the resulting compiled code has no obligation
39 to be LGPL or GPL. For example you are free to choose a commercial or
40 closed source license or any other license if you decide so.
41 *****/
42 *****/
43
44 declare name "Generic Platform Library";
45 declare version "1.3.0";
46
47 //-----'(pl.)SR'-----
48 // Current sampling rate (between 1 and 192000Hz). Constant during
49 // program execution. Setting this value to a constant will allow the
50 // compiler to optimize the code by computing constant expressions at
51 // compile time, and can be valuable for performance, especially on
52 // embedded systems.
53 //-----
54 SR = min(192000.0, max(1.0, fconstant(int fSamplingFreq, <math.h>)));
55
56 //-----'(pl.)BS'-----
57 // Current block-size (between 1 and 16384 frames). Can change during the execution.
58 //-----
59 BS = min(16384.0, max(1.0, fvariable(int count, <math.h>)));
60
61 //-----'(pl.)tablesize'-----
62 // Oscillator table size. This value is used to define the size of the
63 // table used by the oscillators. It is usually a power of 2 and can be lowered
64 // to save memory. The default value is 65536.
65 //-----
66 tablesize = 1 << 16;

```